Week 1 - Domain 1: SDLC Automation - CI/CD Building Blocks

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Topic	What it covers
1. AWS CodeCommit	Repositories, authentication, IAM, branch protection, triggers, notifications, EventBridge, cross-account access
2. AWS CodeBuild	buildspec.yml phases, environment variables, Parameter Store / Secrets Manager, caching, VPC builds, reports
3. AWS CodeArtifact	Domains, repositories, upstreams, origin controls, npm / pip / Maven, cross-account, VPC endpoints

Part 1: AWS CodeCommit

1.1 What CodeCommit is

- **Managed private Git hosting.** AWS runs it, so there are no servers to manage, it scales on its own, and it is highly available.
- Your code stays inside AWS and is controlled by **IAM**, logged by **CloudTrail**, and encrypted by **KMS**.
- **Encryption at rest:** always on, using KMS. The default is the AWS-managed key `aws/codecommit`; you can choose a **customer-managed key** instead.
- **Encryption in transit:** HTTPS or SSH.
- **Status note:** AWS stopped onboarding new customers in July 2024 and later restored full availability (check the current docs). It is **still in scope for DOP-C02**.

1.2 Authentication: how you connect

Method	Who uses it	Key points
HTTPS Git credentials	IAM users	A static username and password generated in IAM. Simplest, but long-lived.
SSH keys	IAM users	Upload a public key to your IAM user; the SSH key ID is the username.
git-remote-codecommit (GRC)	IAM users, roles, SSO / federated users	Uses **temporary credentials** from an AWS CLI profile. URL: `codecommit::us-east-1://profile@repo`. **Recommended.**
AWS CLI credential helper	Roles and federated users	Older way to use temporary credentials over HTTPS.

Exam traps:

- The **root user cannot** use Git credentials or SSH keys with CodeCommit.
- Federated, SSO, or cross-account users must use **GRC or the credential helper**, because they have no IAM user to hold Git credentials or SSH keys.

1.3 Authorization: IAM only

- CodeCommit has **no resource-based policies on repositories** (unlike S3 bucket policies). Very common exam point.
- All access is controlled by **identity-based IAM policies**. Repository ARN: `arn:aws:codecommit:us-east-1:111122223333:MyRepo`.
- AWS managed policies:
  - `AWSCodeCommitFullAccess` - everything, including deleting repositories.
  - `AWSCodeCommitPowerUser` - everything **except deleting repositories**. The usual developer policy.
  - `AWSCodeCommitReadOnly` - read-only access.

1.4 Branch protection

There is no "protected branch" checkbox like GitHub has. You combine two things.

a) An IAM Deny policy on specific branches - the condition key `codecommit:References` matches the branch name. Attach it to the developers' group:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": [
      "codecommit:GitPush",
      "codecommit:DeleteBranch",
      "codecommit:PutFile",
      "codecommit:Merge*"
    ],
    "Resource": "arn:aws:codecommit:us-east-1:111122223333:MyRepo",
    "Condition": {
      "StringEqualsIfExists": { "codecommit:References": ["refs/heads/main"] },
      "Null": { "codecommit:References": "false" }
    }
  }]
}
```

- The `Null` block makes the Deny apply only when a branch reference is present in the request.
- Leads and the pipeline role do **not** get this policy, so they can still merge.

b) Pull request approval rules

- An **approval rule** applies to a single pull request.
- An **approval rule template** is reusable: associate it with repositories and it applies to new pull requests targeting the branches you list.
- Example: "2 approvals from members of the `SeniorDevs` IAM role" before a merge is allowed. Approvers can be IAM users or an **approval pool** of roles with wildcards.

Typical exam answer: "Prevent direct pushes to main and require 2 reviewers" = **IAM Deny using** `codecommit:References` + **approval rule template**.

1.5 Triggers, notifications, and EventBridge

The most-tested part of CodeCommit. Learn the differences:

Feature	Events it reacts to	Targets	Use it when
Triggers (repository setting)	Only repository changes: push to a branch, create / delete branch or tag	SNS, Lambda (max 10 triggers per repository)	A simple action when someone pushes
Notification rules (CodeStar Notifications)	Pull requests (created, updated, merged), comments, approvals, branches and tags	SNS, Slack / Teams via AWS Chatbot (Amazon Q Developer in chat applications)	Telling **people** what happened
Amazon EventBridge	**All** CodeCommit events: referenceCreated / Updated / Deleted, pullRequestCreated, PR state changes, comments	Any EventBridge target: Lambda, CodeBuild, Step Functions, SQS, CodePipeline	Automation, especially for **pull request events**

Exam rules of thumb:

- **Run a build or test when a pull request is created** (CodeBuild posts the result as a PR comment): use **EventBridge**. Triggers cannot see pull request events.
- **Start CodePipeline from CodeCommit:** use an **EventBridge rule** (default and recommended). Periodic polling is legacy.
- **Notify a Slack channel about pull requests:** use **notification rules** with Chatbot.
- **Copy a repository to another Region or account:** there is **no built-in replication**. Use EventBridge to run CodeBuild or Lambda that does `git push --mirror`.

1.6 Cross-account access

Scenario: the repository is in **Account A**, the developers are in **Account B**.

1. In **Account A**, create an IAM role, for example `CodeCommitCrossAccountRole`:
   - **Trust policy:** allows Account B (`arn:aws:iam::<AccountB>:root`).
   - **Permissions policy:** CodeCommit access to that repository only.
2. In **Account B**, grant the developers' group `sts:AssumeRole` on that role's ARN.
3. Developers configure an AWS CLI profile that uses the role, then clone with GRC:

```
git clone codecommit::us-east-1://crossaccount-profile@MyRepo
```

Remember: there is no repository policy, so cross-account access **always means assuming a role**.

Cross-account CodePipeline (pipeline in a "Tools" account, repository in a "Dev" account) needs:

- A role in the source account that the pipeline can assume.
- A **customer-managed KMS key** for the artifact bucket (the default key cannot be shared across accounts).
- An S3 bucket policy that lets the other account's role read and write artifacts.

1.7 Other exam points

- **Monitoring:** CloudTrail records CodeCommit API calls, including `GitPush` and `GitPull`. Use it for "who deleted the branch?" questions.
- **Code review:** attach **CodeGuru Reviewer** to a repository for automatic comments on pull requests.
- **Migrating from GitHub or Bitbucket:** `git clone --mirror`, then `git push --mirror` to CodeCommit.
- **Large files:** files over about 6 MB can't be added through the console or API; use Git LFS or store them in S3.
- **Permissions for targets:** an SNS topic policy allowing `codestar-notifications.amazonaws.com`, and a Lambda resource policy allowing `codecommit.amazonaws.com`.

1.8 Hands-on lab (about 1.5 hours)

1. Create a repository called `dop-lab-repo`.
2. Install git-remote-codecommit (`pip install git-remote-codecommit`) and clone with `git clone codecommit::us-east-1://dop-lab-repo`.
3. Create an IAM group `Developers` with `AWSCodeCommitPowerUser` and the Deny-on-main policy. Check that pushing to `main` fails and pushing to `feature/x` works.
4. Create an approval rule template requiring 1 approval for `main` and associate it with the repository. Try to merge a pull request without approval.
5. Add a trigger that invokes a Lambda function on push; log the event.
6. Create an EventBridge rule for `pullRequestCreated` that publishes to an SNS email topic.
7. Create a notification rule that sends pull request comments to SNS.
8. Clean up: delete the repository, Lambda function, SNS topic, and rules.

1.9 Check yourself

1. A developer signs in through IAM Identity Center and needs to clone a repository. Which authentication method should they use?
2. You must run unit tests automatically when a pull request is created and post the result as a comment. Trigger or EventBridge, and why?
3. Can you grant another account access by attaching a policy to the repository? If not, what do you do?

Answers:

1. **git-remote-codecommit** (or the credential helper) - federated users have no IAM user for Git credentials or SSH keys.
2. **EventBridge.** Triggers only fire on repository events (pushes, branch changes), not pull request events. The rule starts CodeBuild, and Lambda posts the result with `PostCommentForPullRequest`.
3. **No.** CodeCommit has no repository policies. Create a role in the repository's account that trusts the other account, and have the users assume it.

Part 2: AWS CodeBuild

2.1 What CodeBuild is

- **A fully managed build service.** Compiles code, runs tests, and produces artifacts or Docker images. No build servers (unlike Jenkins), scales automatically, **pay per build minute**.
- Each build runs in a **fresh, temporary container**. Nothing carries over between builds unless you use caching.
- **Build images:** AWS-managed images (Amazon Linux, Ubuntu, Windows, ARM, GPU), or **custom images** from ECR / Docker Hub when you need special tools.
- **Compute types:** small, medium, large, 2xlarge, plus Lambda compute for fast short builds. **Reserved capacity fleets** keep machines warm.
- **Timeouts:** build timeout defaults to **60 minutes**, maximum **8 hours**. A separate **queued timeout** limits how long a build waits in the queue.
- **Sources:** CodeCommit, GitHub, GitHub Enterprise, Bitbucket, GitLab, S3, or the CodePipeline artifact.
- **Starting builds automatically:**
  - **Webhooks** work for **GitHub, Bitbucket, and GitLab**, with **filter groups** (event `PULL_REQUEST_CREATED`, branch `^refs/heads/feature/.*`, file path).
  - **CodeCommit has no webhooks** - use an **EventBridge rule** or CodePipeline.

2.2 buildspec.yml: the build instructions

- Default location: `buildspec.yml` in the **root of the source code**.
- Override with a different file name or path (for example `buildspecs/test.yml`) or an inline buildspec - this is how you run **several build projects from one repository**.

```yaml
version: 0.2

run-as: root                # optional: Linux user that runs the commands

env:
  shell: bash
  variables:                # plain-text values (NOT for secrets)
    APP_ENV: "staging"
  parameter-store:          # read from SSM Parameter Store
    DB_HOST: "/myapp/staging/db_host"
  secrets-manager:          # read from Secrets Manager
    DB_PASSWORD: "prod/myapp/db:password"
  exported-variables:       # pass values on to later CodePipeline stages
    - IMAGE_TAG
  git-credential-helper: yes

phases:
  install:
    runtime-versions:
      python: 3.12
      nodejs: 20
    commands:
      - pip install -r requirements.txt
  pre_build:
    commands:
      - aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin
    $ECR_URI
      - export IMAGE_TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
  build:
    on-failure: ABORT        # ABORT (default) or CONTINUE
    commands:
      - pytest --junitxml=reports/junit.xml
      - docker build -t $ECR_URI:$IMAGE_TAG .
    finally:
      - echo "runs even if the commands above fail"
  post_build:
    commands:
      - |
        if [ "$CODEBUILD_BUILD_SUCCEEDING" = "1" ]; then
          docker push $ECR_URI:$IMAGE_TAG
        fi
      - printf '[{"name":"web","imageUri":"%s"}]' $ECR_URI:$IMAGE_TAG > imagedefinitions.json

reports:
  pytest-reports:
    files: ["junit.xml"]
    base-directory: reports
    file-format: JUNITXML

artifacts:
  files:
    - imagedefinitions.json
    - appspec.yml
    - "scripts/**/*"
  discard-paths: no

cache:
  paths:
    - "/root/.cache/pip/**/*"
    - "node_modules/**/*"
```

Phase order:

```
SUBMITTED > QUEUED > PROVISIONING > DOWNLOAD_SOURCE > INSTALL > PRE_BUILD
          > BUILD > POST_BUILD > UPLOAD_ARTIFACTS > FINALIZING > COMPLETED
```

Phase	What to put in it
install	Install tools, choose runtime-versions
pre_build	Log in to ECR, install dependencies, lint, set variables
build	Compile, run unit tests, docker build
post_build	Push the image, package output, send notifications

Behavior the exam likes to ask about:

- If **install or pre_build fails**, the build stops - build and post_build are skipped.
- If **build fails, post_build still runs.** Check `CODEBUILD_BUILD_SUCCEEDING` (1 = success so far, 0 = failure) so you don't push a broken image.
- A `finally` block runs even if the commands before it in that phase failed.
- `on-failure: CONTINUE` lets the build carry on to the next phase after a failure.

2.3 Environment variables

Precedence (highest first):

1. Values passed when the build is started (`start-build --environment-variables-override`, or the CodePipeline action configuration).
2. Values in the build project's environment settings.
3. Values in the buildspec `env` section.

Project variable types: `PLAINTEXT`, `PARAMETER_STORE`, `SECRETS_MANAGER`.

Exam trap: never put secrets in `PLAINTEXT` - plain-text values are visible in the console, API output, and CloudTrail.

Built-in variable	Meaning
CODEBUILD_RESOLVED_SOURCE_VERSION	The commit ID that was built (often the image tag)
CODEBUILD_SOURCE_VERSION	The branch, commit, PR, or S3 object version requested
CODEBUILD_BUILD_ID / CODEBUILD_BUILD_NUMBER	Unique build ID / increasing build number
CODEBUILD_BUILD_SUCCEEDING	1 or 0 - use it in post_build
CODEBUILD_WEBHOOK_TRIGGER	What started the build, e.g. pr/12 or branch/main
CODEBUILD_SRC_DIR	Where the source code is checked out
AWS_REGION, AWS_DEFAULT_REGION	The Region the build runs in

Passing values to later pipeline steps: `exported-variables` makes a value (e.g. `IMAGE_TAG`) available to later CodePipeline actions through a **variable namespace**, e.g. `#{BuildVariables.IMAGE_TAG}`.

2.4 Secrets: Parameter Store and Secrets Manager

```
env:
  parameter-store:
    API_URL: "/myapp/prod/api_url"            # String or SecureString
  secrets-manager:
    DB_PASS: "prod/db-creds:password"          # secret-id:json-key:version-stage:version-id
    API_KEY: "arn:aws:secretsmanager:us-east-1:111122223333:secret:apikey-AbCd"
```

The build's service role needs:

- `ssm:GetParameters` on the parameter.
- `secretsmanager:GetSecretValue` on the secret.
- `kms:Decrypt` - only if encrypted with a **customer-managed key**.

Other points:

- Values are **masked in build logs**.
- **Secrets Manager** = automatic **rotation** (e.g. RDS passwords) or **cross-account sharing**. **Parameter Store** = cheaper, for config values and simple secrets.
- To **pull a private build image** from Docker Hub, set the project's **registry credential** to a Secrets Manager secret. To run `docker login` inside the build, read the password from Secrets Manager into an environment variable.

2.5 Caching

Cache type	Where it lives	How it works	Use it for
Amazon S3 cache	S3 bucket	Downloaded at build start, uploaded at end. Lasts and is shared across hosts.	Dependencies when builds don't run often
Local - source cache	Build host	Keeps Git metadata so only changes are fetched	Large repositories
Local - Docker layer cache	Build host	Reuses Docker image layers. Requires privileged mode.	Docker image builds
Local - custom cache	Build host	Keeps folders listed under cache.paths	Fast builds that run often

- The **local cache is best-effort** - only helps when builds run often on the same host; not guaranteed.
- **S3 caching is reliable but slower** to restore.
- "Docker builds are slow": **local Docker layer cache**, `docker build --cache-from <ECR image>`, or a **custom build image** with tools preinstalled.
- **Building Docker images** inside CodeBuild needs **privileged mode**.

2.6 Builds inside a VPC

- **Why:** reach **private resources** - RDS, ElastiCache, internal APIs, private CodeArtifact / Nexus - usually for integration tests.
- **Setup:** in the project, choose the **VPC ID**, **private subnets**, and **security groups**.
- CodeBuild creates an **ENI** in your subnet with **no public IP**. Even in a public subnet it **cannot reach the internet**.
- Internet access (pip, npm, Docker Hub) needs a **NAT gateway** from a **private subnet**.
- Or avoid the internet with **VPC endpoints**:

Interface endpoints	Gateway endpoint
ecr.api, ecr.dkr, logs, ssm, secretsmanager, sts, codeartifact.api, codeartifact.repositories	S3 (also needed for ECR image layers)

- **Service role EC2 permissions:** `ec2:CreateNetworkInterface`, `ec2:DescribeNetworkInterfaces`, `ec2:DeleteNetworkInterface`, `ec2:DescribeSubnets`, `ec2:DescribeSecurityGroups`, `ec2:DescribeDhcpOptions`, `ec2:DescribeVpcs`, `ec2:CreateNetworkInterfacePermission`.

Classic exam scenario: "After setting the VPC configuration, builds fail to download dependencies or time out reaching S3." Fix: **private subnets with a NAT gateway**, or **VPC endpoints**. Moving to a public subnet does **not** fix it.

2.7 Test and code coverage reports

Report type	Supported formats
Test reports	JUnit XML, Cucumber JSON, TestNG XML, NUnit / NUnit3 XML, Visual Studio TRX
Code coverage	JaCoCo XML, Cobertura XML, Clover XML, SimpleCov JSON, LCOV

- **Report groups** collect results from each build and show pass/fail trends in the console.
- Configure in the buildspec `reports:` section, or point at an existing report group ARN.
- Report data is kept for **30 days**; **export raw files to S3** (optionally KMS-encrypted) to keep them longer.
- **Service role needs:** `codebuild:CreateReportGroup`, `codebuild:CreateReport`, `codebuild:UpdateReport`, `codebuild:BatchPutTestCases`, `codebuild:BatchPutCodeCoverages`.

2.8 Logging, monitoring, debugging, and batch builds

- **Logs:** CloudWatch Logs (default), S3, or both.

- **CloudWatch metrics:** `Builds`, `SucceededBuilds`, `FailedBuilds`, `Duration`, `QueuedDuration` - you can alarm on these.
- **EventBridge events:** "CodeBuild Build State Change" (IN_PROGRESS, SUCCEEDED, FAILED, STOPPED) and "Build Phase Change" - for notifications and automation.
- **Notification rules** send messages to SNS or Slack.
- **Debugging:** Session Manager into a running build (`codebuild-breakpoint` pauses it); the **CodeBuild local agent** runs a buildspec on your machine.
- **Batch builds:** `build-list` (parallel), `build-matrix` (multiple runtimes / OS / variables), `build-graph` (dependent builds).

2.9 Exam traps: quick table

Scenario	Answer
Database password visible in the console	Move it to a SECRETS_MANAGER / PARAMETER_STORE variable
Secret must rotate automatically	Secrets Manager
Build when a CodeCommit pull request is opened	EventBridge rule -> CodeBuild (no CodeCommit webhooks)
Build on a GitHub PR for feature/* only	Webhook with filter groups
"Cannot connect to the Docker daemon"	Turn on privileged mode
Build in a VPC can't reach the internet	Private subnet with NAT gateway, or VPC endpoints
Slow builds - dependencies download every time	S3 cache or local custom cache
Pass the image tag to a later deploy stage	exported-variables + CodePipeline variable namespace
Don't push the image if tests failed	Check CODEBUILD_BUILD_SUCCEEDING in post_build
Test pass/fail trends over time	Report groups (JUnit, etc.)
Special tools baked into the build	Custom build image in ECR

2.10 Hands-on lab (about 2 hours)

1. Create a small Python app with `pytest` tests and push it to your CodeCommit repository.
2. Create a CodeBuild project: managed Amazon Linux image, **privileged mode** on, CloudWatch Logs on.
3. Store a fake secret `dop/lab/api:key` in Secrets Manager and a parameter `/dop/lab/env` in Parameter Store.
4. Write a buildspec that reads both values, runs `pytest --junitxml`, adds a `reports:` section, builds a Docker image, and pushes it to ECR tagged with the commit ID.
5. Run the build. Confirm the secret shows as `***` in the logs and the test report appears in the console.
6. Add an S3 cache for pip. Run twice and compare the install phase duration.
7. Break it on purpose: make a test fail (post_build runs but the image is not pushed); turn privileged mode off and read the Docker error.
8. Optional: put the build in a private subnet without a NAT gateway and watch it fail; then add NAT or endpoints.
9. Clean up: ECR images, NAT gateway (hourly cost), secret, project.

2.11 Check yourself

1. The build phase fails. Does post_build run? How do you stop the image from being pushed?
2. A CodeBuild project was moved into a VPC to reach RDS. Now `npm install` times out. Two possible fixes?
3. Which setting is required to run `docker build` inside CodeBuild?
4. How do you pass the Docker image tag from CodeBuild to a later deploy action in CodePipeline?
5. The same variable is set in the buildspec, the project, and at build start. Which wins?

Answers:

1. **Yes, post_build still runs.** Wrap the push in `if [ "$CODEBUILD_BUILD_SUCCEEDING" = "1" ]`.

2. **Private subnets with a NAT gateway**, or a **CodeArtifact npm proxy with VPC endpoints**. A public subnet will not work.
3. **Privileged mode.**
4. **exported-variables** with a namespace on the build action, then `#{Namespace.IMAGE_TAG}`. For ECS you can also write the image into `imagedefinitions.json`.
5. The value **passed at build start**, then the project, then the buildspec.

Part 3: AWS CodeArtifact

3.1 What CodeArtifact is

- **A managed, private package repository** - a private npm, PyPI, or Maven Central inside your AWS account. Replaces self-hosted Nexus, Artifactory, or verdaccio.
- **Two jobs:**
  1. **Stores your company's own packages** (e.g. `@mycorp/utils`, `mycorp-common.jar`).
  2. **Proxies and caches public packages** from npmjs, PyPI, Maven Central - builds keep working if the public site is down, and you control what is allowed.
- **Formats:** npm (yarn, pnpm), Python (pip, twine), Maven / Gradle, NuGet, Swift, Ruby, Cargo, and **generic** packages.
- Encrypted with KMS; access by IAM **and resource policies** (unlike CodeCommit); CloudTrail logging; EventBridge events.

3.2 Domains

- A **domain** is the **top-level container**. Every repository lives inside one domain.
- **Package files are stored once per domain** - the same file in 10 repositories is paid for once.
- **One KMS key per domain**, chosen at creation and **cannot be changed afterwards** (common exam trap).
- A **domain policy** (resource-based) controls **cross-account access**, especially who can get an authorization token.
- The **domain owner account** pays for storage.
- **Best practice:** one domain for the whole organization in a shared "artifacts" / "tools" account, separate repositories per team or environment.

```
Domain: mycorp  (owned by Tools account, KMS key: mycorp-artifacts-key)
 |-- npm-store      (external connection: public:npmjs)
 |-- pypi-store     (external connection: public:pypi)
 |-- maven-store    (external connection: public:maven-central)
 |-- team-a-repo    (upstreams: npm-store, pypi-store)
 '-- release-repo   (approved and promoted packages only)
```

3.3 Repositories

- A **repository** holds **package versions**; one repository can hold several formats.
- A **repository policy** (resource-based) controls who can read or publish, including other accounts.
- An **external connection** links a repository to a public site: `public:npmjs`, `public:pypi`, `public:maven-central`, `public:nuget-org`, `public:crates-io`.
- **Only one external connection per repository** - create one "store" repository per public source.
- **Package version status:** Published, Unfinished, Unlisted, Archived, Disposed. Change status to stop a bad version being used.
- **Promotion:** `copy-package-versions` copies a version from `dev-repo` to `release-repo` after tests pass (often a CodePipeline step).

3.4 Upstream repositories

- A repository can list **up to 10 upstream repositories**, searched **in the order listed**, up to a repository with an external connection.
- A version fetched through upstreams is **kept (copied) in the downstream repository** you requested from, and in the repositories in between. It stays available even if removed from the public site.

```
Developer / CodeBuild
        |   npm install lodash
        v
  team-a-repo  --(upstream)-->  npm-store  --(external connection)-->  registry.npmjs.org
  (lodash kept here)            (lodash kept here)
```

- **Why chain repositories:** one place to control and audit public packages; teams get internal and public packages from **one endpoint**.

Package origin controls (dependency confusion protection):

- **Publish:** ALLOW / BLOCK - can versions be published directly to this repository?
- **Upstream:** ALLOW / BLOCK - can versions come in from upstream repositories?

How the package first arrived	Publish	Upstream	Effect
Your company published it first	ALLOW	BLOCK	Blocks public versions
It first came from upstream	BLOCK	ALLOW	You can't publish versions under that name

- Stops an attacker publishing a public package with the **same name as your internal package** and a higher version ("dependency confusion" / "dependency substitution").
- **Exam keywords:** "malicious public package with same name as internal package" = **package origin controls**.

3.5 Authentication and permissions

- You need a **temporary authorization token** - valid **12 hours by default** (15 minutes to 12 hours).

```
aws codeartifact get-authorization-token --domain mycorp --domain-owner 111122223333 --query
    authorizationToken --output text
```

- `aws codeartifact login` gets a token **and** configures the tool (npm, pip, twine only):

```
aws codeartifact login --tool npm --domain mycorp --domain-owner 111122223333 --repository team-a-repo
```

To do this	You need
Get a token	codeartifact:GetAuthorizationToken AND sts:GetServiceBearerToken (condition sts:AWSServiceName = codeartifact.amazonaws.com)
Download packages	codeartifact:ReadFromRepository, codeartifact:GetRepositoryEndpoint
Publish packages	codeartifact:PublishPackageVersion, codeartifact:PutPackageMetadata

Exam trap: "not authorized to get token" despite CodeArtifact permissions = add `sts:GetServiceBearerToken`.

3.6 Setting up npm, pip, and Maven

npm - writes the registry URL and token into `~/.npmrc` (add `--namespace mycorp` to scope only `@mycorp/*`):

```
aws codeartifact login --tool npm --domain mycorp --domain-owner 111122223333 --repository team-a-repo
npm install
npm publish
```

pip and twine - pip login sets `index-url` in `pip.conf`; twine login sets up `~/.pypirc`:

```
aws codeartifact login --tool pip --domain mycorp --domain-owner 111122223333 --repository team-a-repo
pip install -r requirements.txt

aws codeartifact login --tool twine --domain mycorp --domain-owner 111122223333 --repository team-a-repo
twine upload --repository codeartifact dist/*
```

Maven and Gradle - `aws codeartifact login` does **not** support them; set the token yourself.

```
export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token --domain mycorp --domain-owner
    111122223333 --query authorizationToken --output text)
```

```
<!-- settings.xml -->
<servers>
  <server>
    <id>mycorp-team-a-repo</id>
    <username>aws</username>
    <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
  </server>
</servers>

<!-- pom.xml (URL from: get-repository-endpoint --format maven) -->
<repositories>
  <repository>
    <id>mycorp-team-a-repo</id>
    <url>https://mycorp-111122223333.d.codeartifact.us-east-1.amazonaws.com/maven/team-a-repo/</url>
  </repository>
</repositories>
<distributionManagement>   <!-- for mvn deploy (publishing) -->
  <repository>
    <id>mycorp-team-a-repo</id>
    <url>https://mycorp-111122223333.d.codeartifact.us-east-1.amazonaws.com/maven/team-a-repo/</url>
  </repository>
</distributionManagement>
```

- **Gradle:** a `maven { url ...; credentials { username "aws"; password System.env.CODEARTIFACT_AUTH_TOKEN } }` block.

In CodeBuild (service role needs the permissions in 3.5):

```
phases:
  pre_build:
    commands:
      - aws codeartifact login --tool pip --domain mycorp --domain-owner 111122223333 --repository
    team-a-repo
  build:
    commands:
      - pip install -r requirements.txt
      - python -m build
      - aws codeartifact login --tool twine --domain mycorp --domain-owner 111122223333 --repository
    team-a-repo
      - twine upload --repository codeartifact dist/*
```

3.7 Cross-account access

Scenario: domain and repository in the **Tools account**, builds in the **Dev account**.

1. **Domain policy** (Tools): allow the Dev role `codeartifact:GetAuthorizationToken`.
2. **Repository policy** (Tools): allow the Dev role `codeartifact:ReadFromRepository` (and `PublishPackageVersion` if it publishes).
3. **IAM policy** (Dev): same permissions plus `sts:GetServiceBearerToken`.
4. **KMS:** if the domain uses a customer-managed key, the key policy must allow the other account.

Both sides must allow access: **resource policy** in Tools **and IAM policy** in Dev.

3.8 Using CodeArtifact from a private VPC (no internet)

- Builds in private subnets can use CodeArtifact **without a NAT gateway** - CodeArtifact fetches public packages **on your behalf**.

Endpoint	Type	Notes
com.amazonaws.<region>.codeartifact.api	Interface	API calls and getting tokens
com.amazonaws.<region>.codeartifact.repositories	Interface	npm / pip / maven traffic; turn on private DNS
S3	Gateway	Package files are stored in S3 - allow the CodeArtifact asset bucket
STS	Interface	Only if the build gets credentials through STS

Exam scenario: "Builds in a private subnet with no internet must install public npm packages, without a NAT gateway" = **CodeArtifact external connection + CodeArtifact VPC endpoints + S3 gateway endpoint**.

3.9 Events and automation

- CodeArtifact sends a **package version state change** event to **EventBridge** when a version is published, changed, or deleted.
- "When a new version of the shared library is published, rebuild dependent services" = **EventBridge rule -> CodePipeline or CodeBuild**.

3.10 Exam traps: quick table

Scenario	Answer
Replace self-hosted Nexus / Artifactory with less work	CodeArtifact
Cache public npm / PyPI so builds survive a public outage	Repository with an external connection, used as an upstream
One endpoint for internal and public packages	Team repository with upstreams to store repositories
Dependency confusion / malicious same-name public package	Package origin controls (block upstream for internal names)
Store packages once across many repositories	They share one domain
Change the domain's KMS key	Not possible - create a new domain and migrate
Share packages with other accounts	Domain policy + repository policy + IAM in the other account
Private subnet needs public packages without NAT	CodeArtifact + VPC endpoints (api, repositories) + S3 gateway endpoint
Token request fails despite CodeArtifact permissions	Add sts:GetServiceBearerToken
Promote a tested package from dev to release	copy-package-versions
Rebuild dependents when a library is published	EventBridge rule -> CodePipeline / CodeBuild
Connect one repository to both npmjs and PyPI	Not possible - one external connection per repository; use store repos as upstreams

3.11 Hands-on lab (about 1.5 hours)

1. Create the domain `dop-lab` with the default `aws/codeartifact` key.
2. Create `pypi-store` with an external connection to `public:pypi`.
3. Create `team-repo` with `pypi-store` as its upstream.
4. Run `aws codeartifact login --tool pip ...` for `team-repo`, then `pip install requests`. Check that `requests` appears in **both** `team-repo` and `pypi-store`.
5. Build a tiny Python package `dop-lab-utils` and publish it with twine to `team-repo`.
6. Check the package origin controls for `dop-lab-utils`: publish ALLOW, upstream BLOCK.
7. Update your CodeBuild project to install dependencies through CodeArtifact (add read permissions and `sts:GetServiceBearerToken` to the service role).
8. Create an EventBridge rule that emails you (SNS) when a package version is published; publish version 0.2.
9. Clean up: delete the repositories, then the domain.

3.12 Check yourself

1. Your team needs both PyPI and npm public packages through one repository endpoint. How do you set that up?
2. An attacker publishes `mycorp-auth` 99.0 on public PyPI, with the same name as your internal package. What stops builds pulling it?
3. CodeBuild in private subnets with no NAT gateway must install from CodeArtifact. Which endpoints are required?

4. What two permissions does a role need just to **get a token**?
5. The domain uses the AWS-managed key; security wants a customer-managed key. What do you do?

Answers:

1. Create `npm-store` (`public:npmjs`) and `pypi-store` (`public:pypi`), and add both as **upstreams** of the team repository.
2. **Package origin controls** - the internal package was published directly, so upstream is BLOCK.
3. Interface endpoints **codeartifact.api** and **codeartifact.repositories** (private DNS) plus an **S3 gateway endpoint** (and STS if needed).
4. **codeartifact:GetAuthorizationToken** and **sts:GetServiceBearerToken**.
5. You **can't change a domain's key** - create a new domain with the customer-managed key and migrate the packages.